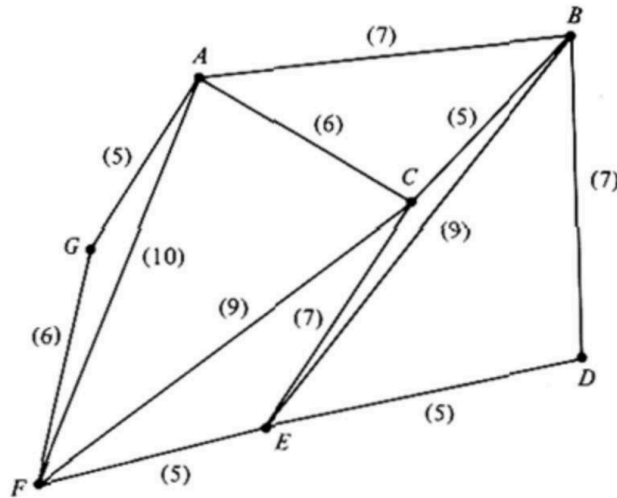


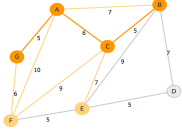
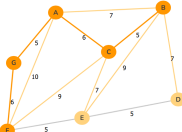
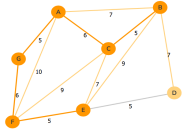
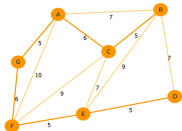
Algorithm Implementation to find MST

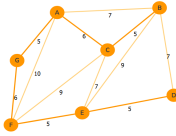


Repo: <https://github.com/nellaarrgghh/Informatics-ITS-Graph-Theory-class-Group-6>

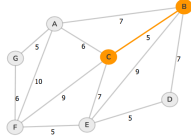
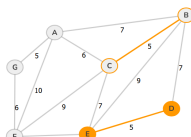
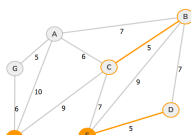
1. Finding MST using Prim's Algorithm

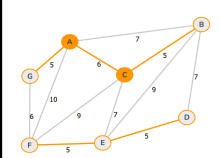
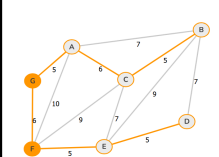
Step	U (Visited)	Edges(u,v)	V - U (Unvisited)	Graph
0	{ }	-	{A, B, C, D, E, F, G}	-
1	{A}	(A-G) = 5 ✓	{B, C, D, E, F, G}	
		(A-C) = 6		
		(A-B) = 7		
		(A-F) = 10		
2	{A, G}	(A-C) = 6 ✓	{B, C, D, E, F}	
		(G-F) = 6		
		(A-B) = 7		
		(A-F) = 10		

3	{A, G, C}	(C-B) = 5✓	{B, D, E, F}	
		(G-F) = 6		
		(A-B) = 7		
		(C-E) = 7		
		(C-F) = 9		
		(A-F) = 10		
4	{A, G, C, B}	(G-F) = 6✓	{D, E, F}	
		(A-B) = 7		
		(C-E) = 7		
		(B-D) = 7		
		(C-F) = 9		
		(B-E) = 9		
5	{A, G, C, B, F}	(F-E) = 5✓	{D, E}	
		(A-B) = 7		
		(C-E) = 7		
		(B-D) = 7		
		(C-F) = 9		
		(B-E) = 9		
6	{A, G, C, B, F, E}	(E-D) = 5✓	{D}	
		(A-B) = 7		
		(C-E) = 7		
		(B-D) = 7		
		(C-F) = 9		

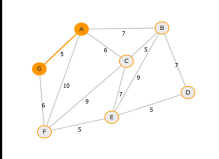
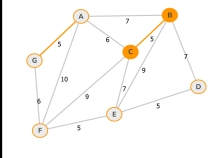
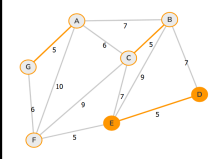
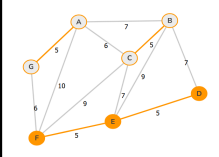
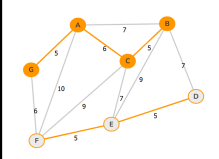
		(B-E) = 9		
		(A-F) = 10		
7	{A, B, C, D, E, F, G}	(A-B) = 7	{ }	
		(C-E) = 7		
		(B-D) = 7		
		(C-F) = 9		

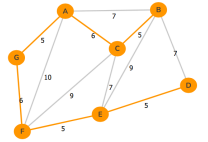
2. Finding MST using Kruskal's Algorithm

Step	Edge(u,v)	Weight	Connected Components	Graph
0	-	-	{{A}, {B}, {C}, {D}, {E}, {F}, {G}}	-
1	(B-C)	5	{{A}, {B, C}, {D}, {E}, {F}, {G}}	
2	(D-E)	5	{{A}, {B, C}, {D, E}, {F}, {G}}	
3	(E-F)	5	{{A}, {B, C}, {D, E, F}, {G}}	
4	(A-G)	5	{{A, G}, {B, C}, {D, E, F}}	

5	(C-A)	6	{{A, B, C, G}, {D, E, F}}	
6	(G-F)	6	{{A, B, C, D, E, F, G}}	

3. Finding MST using Borůvka's Algorithm

Step [Phase]	Active Component	Edge(u,v)	Current Forest	Graph
0 [-]	{ }	-	{{A}, {B}, {C}, {D}, {E}, {F}, {G}}	-
1 [1]	{A}	(A-G) = 5	{{A, G}, {B}, {C}, {D}, {E}, {F}}	
2 [1]	{B}	(B-C) = 5	{{A, G}, {B, C}, {D}, {E}, {F}}	
3 [1]	{D}	(D-E) = 5	{{A, G}, {B, C}, {D, E}, {F}}	
4 [1]	{E}	(E-F) = 5	{{A, G}, {B, C}, {D, E, F}}	
5 [2]	{C}	(C-A) = 6	{{A, B, C, G}, {D, E, F}}	

6 [2]	{G}	$(G-F) = 6$	{{A, B, C, D, E, F, G}}	
-------	-----	-------------	-------------------------	---

4. Error Simulation

The algorithms we implemented successfully traversed all nodes and edges within the provided graph. Because all three algorithms are greedy and evaluate the graph dynamically from scratch, they handle failures automatically:

- If a node or edge fails: They simply skip the missing connection and automatically pick the next cheapest available edge to bypass the failure.
- If the failure cannot be avoided: They detect that it is physically impossible to connect the remaining parts and report the “Error! Cannot Build MST” message implemented in the program instead of crashing.

```

nac@Nailas-MacBook-Pro Informatics-ITS-Graph-Theory-class-Group-6 % python3
kal.py
Enter Nodes: A B C D
Enter Edges: 3
A B 5
B C 5
C A 5
Checking edge A-B with weight 5
  Adding edge A-B to MST (connects components)
Checking edge B-C with weight 5
  Adding edge B-C to MST (connects components)
Checking edge C-A with weight 5
  Skipping edge C-A (creates cycle)
Error! Cannot Build MST
nac@Nailas-MacBook-Pro Informatics-ITS-Graph-Theory-class-Group-6 %

```